

# Contracts — What Came Before

## Introduction

---

Prior to the formation of SG21 there were a number of different potential solutions proposed for contract based programming in the language. With the formation of SG21, a large number of use cases that any future contract solution might need to satisfy have been presented and polled on (see [P1995](#) for the use cases and the initial poll results).

In order to better understand these use cases, a number of people expressed questions about how those use cases might be met with standard C++20 with no language based contracts, and how each of the prior language proposals might satisfy these use cases. We hope this exposition helps in understanding the use cases better, and helps to leverage the work that has gone before into determining our future direction for language based contracts.

## Revision History

---

Differences from first draft:

- The following scores have been adjusted: [dev.reason.confidence](#), [dev.reason.importance](#), [dev.reason.cost](#), [dev.reason.sideeffects](#), [dev.reason.behaviorcontrol](#), [dev.adapt](#), [cppdev.syntax.macros](#), [cppdev.build.legacy](#), [int.control.runtime](#), [teach.standardized](#), [teach.lifecycle](#), [teach.portable](#), [teach.dumbstudents](#), [teach.layering](#)
- The following notes have been adjusted: [cppapi.contracts.exception](#)

## Methodology

---

There are 4 primary language variations that we wish to evaluate for their merits at satisfying the SG21 use cases:

- N4842 - the current C++ working draft - is the closest approximation currently available of what will be the C++20 standard, and is (barring something incredibly surprising happening) going to ship with no explicit contract checking facilities in the language or standard library itself. Many groups around the world have implemented contract checking libraries themselves (and come to the committee to attempt to build on those experiences to get those features into the language) and those libraries are still just as usable for contract checking in C++ today as they have always been. We will reference the BDE libraries (<https://github.com/Bloomberg/bde/>, specifically `bsls_assert` and `bsls_review`, which can be found in <https://github.com/bloomberg/bde/tree/master/groups/bsl/bsls>) for examples of what can be done with no new language support.
- N4820 - The C++ draft prior to the Summer 2019 WG21 meeting in Cologne - was the last draft that had the initially accepted contract checking facility. As with all of the other facilities we will discuss here, this proposal allowed for annotating contracts as boolean predicates within an attribute-like syntax that could be attached to functions as preconditions or postconditions, and put inside functions as assertions. Three basic contract levels were allowed - `default`, `audit`, and `axiom` - with all levels either checked at runtime or introducing undefined behavior if not checked and violated.
- P1429 - This paper primarily focuses on redefining the contract behavior of N4820 in terms of named semantics, and allowing for setting of the various contract levels (`default`, `audit`, and `axiom`) to specific behaviors (in lieu of the combination of build level and continuation mode available in N4820).
- P1607 - This paper proposed removing the contract levels entirely and *only* allowing the setting of specific semantics, with the intent to provide more general build time control using macros until consensus could be reached on how that control should be presented. This option, with no build levels and just four literal semantics, reached consensus in EWG on July 15th 2019, prior to the removal of contracts entirely from the draft.

Rather than enumerate all possible variations that could be chosen, we will focus on comparing P1429 with no literal semantics and P1607 with no contract levels.

For each of these variations, we will assign a value in the range 0-100 for each use case with roughly the following meanings:

- 0 - Explicitly prohibited or fundamentally impossible.
- 25 - Feasible with great extra effort or in a highly limited fashion, no useful support for this use case.
- 50 - Limited support, with possible caveats that would prevent real satisfaction of this use case.
- 75 - Supported sufficiently to be used in a realistic situation.
- 100 - Fully supported with no major caveats.

The goal of these numbers is to give a rough feeling for worth. In the future, we expect to see use cases given some weight (based partly on importance to the community and partly on distinctness from other use cases), and then combined with these numbers to give each solution a total value that can be used to understand how effectively it satisfies the needs of the community.

Note that the long term goal here is not to give a number to say "you must vote for this since it has a higher number" but rather to provide a value that can be used to see "this solution measurably improves support for use cases that I personally might not consider important but that others do see as a priority".

Additionally, a number of features could conceivably be built in a compatible way on top of any of these proposals using a system built with preprocessor macros (and such macros are the only viable way to build a system with no language support). Because of this, some scores are split into two numbers, the first being a judgement on how well the proposal satisfies the use case "out of the box", the second being how the proposal can be used with a macro facility above it to meet the specified use case.

## Use Cases

| # | Code                             | As A      | In Order To       | I Want To                                                                                            | N4842<br>C++20 | N4820<br>Pre-Cologne<br>Draft | P1429<br>Semantic<br>Level<br>Control |
|---|----------------------------------|-----------|-------------------|------------------------------------------------------------------------------------------------------|----------------|-------------------------------|---------------------------------------|
| 1 | <a href="#">dev.reason.knowl</a> | Developer | Reason explicitly | Annotate my program anywhere in the code with my current understanding of its structure or execution | 25             | 90                            | 90                                    |

Without a specific contract checking facility there are still ways to annotate a program with information about program contracts, but those are limited to either comments (w validation or normative effect), the classic C `assert` macro, or larger nonstandard macro-based facilities (with no consistency in representation across different impleme Typical usage would be within a function writing something like this:

```
void sqrt(double n) {
    assert(n >= 0.0);
    BSLS_ASSERT(n >= 0.0);
}
```

All of the contract proposals that were considered introduced the same syntax for expressing an expectation that a certain boolean predicate was true at specified points in These satisfy most common use cases in the wild for contract predicates, although being only boolean predicates makes them fall short of being able to capture many more not runtime checkable aspects of a function contract. These include the ability to express this within a function body as assertions like this:

```
double sqrt(double n) {
    [[ assert : n >= 0.0 ]]
    // ..
}
```

More notably, annotations of expectations about execution could be added on function declarations as well -- something not possible with a solely language based solution .

```
void sqrt(double n) [[ pre : n >= 0.0 ]];
```

|   |                                       |           |                   |                                                                                                                                                                                                   |    |   |   |
|---|---------------------------------------|-----------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|---|---|
| 2 | <a href="#">dev.reason.confidence</a> | Developer | Reason explicitly | Express a spectrum of confidence in my annotations, from "unsure" and asking for validation, to "sure" and asking for some effect to be applied (eg. "maybe", "definitely", "assume" 'something') | 40 | 0 | 0 |
| 3 | <a href="#">dev.reason.importance</a> | Developer | Reason explicitly | Express a spectrum of importance of my annotations, from "critical" (eg. bring the system down) to "minor" (eg. lead to a slower fallback)                                                        | 40 | 0 | 0 |

While BDE's `bsls_assert` does not provide confidence as something that can be used to control contract behavior, it is not hard to envision that such a thing could be built in based solution.

Build-level-only solutions do not capture this at all (where N4820 and P1429 presented `default` , `audit` , and `axiom` as the only extra annotations that could be pu contract, and those were intended to capture cost of checking not confidence).

P1607's literal semantics do not provide this directly, but here again one could envision combining a macro-based solution to choose literal semantics that would capture co importance and compute behavior based on that.

|   |                                 |           |                   |                                                                                                                          |    |    |    |
|---|---------------------------------|-----------|-------------------|--------------------------------------------------------------------------------------------------------------------------|----|----|----|
| 4 | <a href="#">dev.reason.cost</a> | Developer | Reason explicitly | Express a spectrum of expected cost at compile or runtime of my annotations, from "unrunnable" to "expensive" to "cheap" | 40 | 80 | 80 |
|---|---------------------------------|-----------|-------------------|--------------------------------------------------------------------------------------------------------------------------|----|----|----|

A macro-based solution would again be able to be built to use an expression of the cost of checking to determine how a contract will be treated, and BDE does exactly this v following variations on how a contract can be checked:

```
void foo(int x) {
    BSLS_ASSERT_OPT(x != 0);
}
```

```

BSLS_ASSERT(x < Utils::getMaxXValue());
BSLS_ASSERT_SAFE(factorial(x) < Utils::getMaxXFactorial());
}

```

The `default`, `audit`, and `axiom` levels captured this similarly:

```

void foo(int x) {
  [[ assert default : x != 0 ]];
  [[ assert audit : x < Utils::getMaxValue() ]];
  [[ assert axiom : is_reachable( staticranges[x].begin(),
                                staticranges[x].end() ) ]];
}

```

Similar to the case for confidence and importance, any system that was built on top of literal semantics for P1607 with macros would be able to choose to capture cost as part of control of contract behavior.

|   |                                     |           |                         |                                                                                        |    |    |    |
|---|-------------------------------------|-----------|-------------------------|----------------------------------------------------------------------------------------|----|----|----|
| 5 | <a href="#">dev.reason.behavior</a> | Developer | Reason about executions | Have annotations affect the execution of my program in accordance with my expectations | 25 | 75 | 90 |
|---|-------------------------------------|-----------|-------------------------|----------------------------------------------------------------------------------------|----|----|----|

N4820 contracts allowed for contract checks to either introduce undefined behavior or be checked at runtime.

P1429 and P1607 provide more flexibility by exposing the choice of 4 different semantics for contracts, with the primary difference being the granularity of that control.

|   |                                        |           |                         |                                                                                                      |     |   |    |
|---|----------------------------------------|-----------|-------------------------|------------------------------------------------------------------------------------------------------|-----|---|----|
| 6 | <a href="#">dev.reason.sideeffects</a> | Developer | Reason about executions | Ensure annotations do not substantially change the meaning of my program whether enabled or disabled | 100 | 0 | 50 |
|---|----------------------------------------|-----------|-------------------------|------------------------------------------------------------------------------------------------------|-----|---|----|

The primary meaning of this use case is that there should be a way to use the contract facility (on or off) without undefined behavior. This does not preclude the ability to introduce undefined behavior - but it needs to be optional. N4820 fails to provide this. A macro-based facility, barring bugs, should be able to easily provide this. P1429 and P1607 provide more flexibility by exposing the choice of 4 different semantics for contracts, with the primary difference being the granularity of that control.

Both P1429 and P1607 did not fix the fact that any side effects in a predicate are treated as undefined behavior. Note that [P1670R0](#) was scheduled for Cologne which had a proposal for an attempt to address that.

|   |                                            |           |                         |                                                                                                                      |    |    |    |
|---|--------------------------------------------|-----------|-------------------------|----------------------------------------------------------------------------------------------------------------------|----|----|----|
| 7 | <a href="#">dev.reason.behaviorcontrol</a> | Developer | Reason about executions | Have the effect of annotations on execution be user controllable (based on whatever aspects, if any, are available). | 65 | 50 | 55 |
|---|--------------------------------------------|-----------|-------------------------|----------------------------------------------------------------------------------------------------------------------|----|----|----|

A macro-based solution can be built to use command line and in-code annotations to determine contract behavior. P1607 and a macro-only facility would both be able to provide behavior control that users might wish to build, with more consistency to the behaviors that different options might give with P1607.

N4820 and P1429 contract control is much more limited, primarily being only doable at the build level of granularity.

|   |                           |           |                                    |                                                                                                     |    |    |    |
|---|---------------------------|-----------|------------------------------------|-----------------------------------------------------------------------------------------------------|----|----|----|
| 8 | <a href="#">dev.adapt</a> | Developer | Adapt and progress with my project | Be able to easily change my confidence, importance, or other properties of my annotations over time | 40 | 25 | 50 |
|---|---------------------------|-----------|------------------------------------|-----------------------------------------------------------------------------------------------------|----|----|----|

The build-level supporting options (N4820 and P1429) allow changing the one attribute they expose (cost). The macro-based solutions would have to build support for this, put that in the source code and thus make editing the attributes of an annotation easy to edit.

An important thing to note is that safely changing annotation properties in released software is equivalent to introducing that contract freshly in any deployments that ignore the old contract and check the new 'level'. The ability to introduce the check at the new 'level' with a continuing semantic is essential to safely adding any checks to already deployed software. P1429 both only allow the control of continuation at a very high level, thus forcing a user to turn continuation on for all existing checks in order to safely add checks that can be added later. P1607 provides the granularity to change this decision on a per-contract basis.

|    |                                        |               |                                    |                                                                         |    |    |    |
|----|----------------------------------------|---------------|------------------------------------|-------------------------------------------------------------------------|----|----|----|
| 9  | <a href="#">dev.readable.syntax</a>    | Developer     | Have readable annotations          | Have annotations with a succinct and elegant syntax                     | 40 | 80 | 80 |
| 10 | <a href="#">dev.parsable</a>           | Developer     | Interoperate with tools or persons | A syntax that can both be parsed and can be reasoned about semantically | 40 | 75 | 75 |
| 11 | <a href="#">cppdev.syntax.familiar</a> | C++ Developer | Get up to speed                    | Have annotations use familiar syntax                                    | 50 | 75 | 75 |
| 12 | <a href="#">cppdev.syntax.cpp</a>      | C++ Developer | Get up to speed                    | Have annotations use C++ syntax                                         | 50 | 75 | 75 |
| 13 | <a href="#">cppdev.syntax.reuse</a>    | C++ Developer | Reuse code                         | Have annotations use my custom types or functions                       | 75 | 75 | 75 |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|-------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|-----|
| 14                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">cppdev.location</a>           | C++ Developer     | Have a single source of truth      | Use same source file for both code and annotations                                                                                                                 | 100 | 100 | 100 |
| <p>Obviously N4820 provides no explicit syntax and is limited to macros, but that still allows for using readable C++ in the annotations.</p> <p>P1607 likely requires macros to facilitate any global controls over behavior, so is a slightly worse syntactic option than what N4820 and P1429 provide (which have identical</p>                                                                                                                                                                                                                                                                                        |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |
| 15                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">dev.readable.keywords</a>     | Developer         | Have readable annotations          | Have annotation keywords or names with intuitive, clear, and unambiguous meanings                                                                                  | 10  | 50  | 50  |
| <p>Given that it provides no new keywords for contracts, readability is going to be entirely dependent on what macro based facility is being used with N4842.</p> <p>For N4820 and P1429, the clarity of 'default' is fairly opaque, and a huge amount of reflector discussion and contention has revolved around the meaning and use of axiom poorly here.</p> <p>P1607 removes the contentious keywords, but introduces new bespoke keywords that while clear are not necessarily intuitive to all, along with needing to build any other cc of those with macros, so it seems only slightly better in this regard.</p> |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |
| 16                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">dev.readable.priority</a>     | Developer         | Have readable annotations          | Have my contract specification to be visually primary, and secondary information (syntax, hints, roles, levels, etc.) to not be distracting                        | 15  | 10  | 10  |
| <p>All of the attribute-based solutions put the meta-information first before the expression, counter to what this use case is asking for. A macro-based solution could choose to pass information differently, but would also be restricted in how the information gets passed into the macro.</p>                                                                                                                                                                                                                                                                                                                       |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |
| 17                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">dev.tooling</a>               | Developer         | Interoperate with tools or persons | Expose annotations to tools that might leverage them (eg. code linter, static analyzer, semantic prover, compiler sanitizer, binary analyzer, code reviewer, etc.) | 0   | 75  | 75  |
| <p>Entirely macro-based facilities would need to be ubiquitous enough for tooling vendors to choose to support them explicitly, along with those tooling vendors needing to work around difficulties of managing pre-preprocessor based analysis.</p>                                                                                                                                                                                                                                                                                                                                                                     |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |
| 18                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">cppdev.syntax.macros</a>      | C++ Developer     | Support modern features            | Minimize use of macros                                                                                                                                             | 0   | 50  | 60  |
| <p>Obviously not very much can be done with the base language without macros.</p> <p>P1607 would require macros to support any form of global controls, so it gets the lowest rating for this use case, while P4820 would be usable for basic applications without P1429 would be slightly more flexible without needing to resort to macros.</p>                                                                                                                                                                                                                                                                         |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |
| 19                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">cppdev.modules</a>            | C++ Developer     | Support modern features            | Be interoperable with modules                                                                                                                                      | 10  | 25  | 25  |
| <p>A purely macro based solution would have the worst interaction with modules.</p> <p>The language-based solutions were expected to have basic support for modules, since all would have landed in C++20, but no specific modules-related features were initial</p>                                                                                                                                                                                                                                                                                                                                                      |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |
| 20                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">cppdev.coroutines</a>         | C++ Developer     | Support modern features            | Be interoperable with coroutines                                                                                                                                   | 25  | 25  | 25  |
| 21                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">api.coroutines</a>            | C++ API Developer | Use coroutines                     | Define and check pre and post conditions as I would a regular function                                                                                             | 0   | 0   | 0   |
| 22                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">api.coroutines.invariants</a> | C++ API Developer | Use coroutines                     | Define and check invariants over all entry and exit points from a coroutine (to its awaiter or promise)                                                            | 0   | 0   | 0   |
| <p>All of the solutions would work perfectly fine within the body of a coroutine. Coroutine handling of contracts on inputs, outputs, and states when resuming a coroutine had no discussions or support planned in the proposals as they existed, so none of the proposals support coroutines any better than a purely macro-based solution.</p>                                                                                                                                                                                                                                                                         |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |
| 23                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <a href="#">cppdev.concepts</a>           | C++ Developer     | Support modern features            | Be interoperable with concepts                                                                                                                                     | 25  | 25  | 25  |
| <p>Nothing about the prior facilities seems to benefit or hinder the use of concepts.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                           |                   |                                    |                                                                                                                                                                    |     |     |     |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                               |                       |                                      |                                                                                                                                               |    |    |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|-----------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|
| 24                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">cppdev.existing.std</a>           | C++ Developer         | Use the standard library in-contract | Codify existing exposition-only standard library requirements                                                                                 | 50 | 50 | 50 |
| Many of the preconditions and postconditions of the standard library can be expressed as boolean expressions, and so are encodable using the facilities that were provided                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                               |                       |                                      |                                                                                                                                               |    |    |    |
| 25                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">cppdev.debugger</a>               | C++ Developer         | Use Debugger                         | Have runtime able to launch a debugger from an annotation if necessary                                                                        | 25 | 40 | 40 |
| <p>As much as any C++ program is able to do something like signal that a debugger should be opened, a macro based solution could invoke the platform-specific functions that enable that (such as <code>__debugbreak()</code> or <code>raise(SIGTRAP)</code>). All of the prior solutions also provided a consistent place to put such a function call by allowing for a set violation handler, and due to that being globally consistent we rate them slightly higher.</p> <p>This is still not a standardized facility, so no solution completely satisfies this use case within the standard itself.</p> |                                               |                       |                                      |                                                                                                                                               |    |    |    |
| 26                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">cppdev.build.legacy</a>           | C++ Developer         | Use existing build modes             | Have annotations affect executions depending on my existing build modes (eg. Debug or Release modes in VS)                                    | 25 | 0  | 0  |
| Those solutions that would use macros to do global control (N4842 and P1607) would be able to reference <code>_NDEBUG</code> or similar macros usually associated with "debug" and builds as input to how they configure contracts. The standard, and the build levels specified in N4820 and P1429, are however completely disconnected from one another.                                                                                                                                                                                                                                                  |                                               |                       |                                      |                                                                                                                                               |    |    |    |
| 27                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">cdev.contracts</a>                | C Developer           | Write contracts on my functions      | Specify contracts in a way standardizable as part of the C language                                                                           | 10 | 0  | 0  |
| 28                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">cdev.identifiers</a>              | C Developer           | Write contracts on my functions      | Use contracts with macro-safe keywords that are reserved C names (i.e., <code>_Pre</code> , <code>_Post</code> , <code>_Assert</code> , etc.) | 10 | 0  | 0  |
| <p>None of the contract specifications as a language feature would have been palatable to the C standards committee, since they all require being able to attach meaning to as well as make use of new identifiers with special meaning (which are unfriendly to standardize in the macro-heavy C world.)</p> <p>A pure C++ macro based contract solution would, however, possibly be implementable in a fashion that is equally usable from C code.</p>                                                                                                                                                    |                                               |                       |                                      |                                                                                                                                               |    |    |    |
| 29                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">cdev.violationhandler</a>         | C Developer           | Write contracts on my functions      | Have a common violation handler for both violated C and C++ contracts                                                                         | 50 | 25 | 25 |
| <p>With no C interoperability easily possible, and the violation handler specified in terms of an opaque class type (<code>std::contract_violation</code>), implementing that violation would not have been feasible.</p> <p>It is, however, likely that any C-based contract solution would be able to bridge to and from the C++ contract violation handler, since there is no particularly complex function violation handler or the violation object itself.</p>                                                                                                                                        |                                               |                       |                                      |                                                                                                                                               |    |    |    |
| 30                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">cdev.ignorable</a>                | C Developer           | Write contracts on my functions      | Make all contract semantics optional (so as not to change WG14-N2385 6.7.11 p2)                                                               | 50 | 25 | 25 |
| <p>P1607's explicit semantics require that they behave in a specific way when asked for. Similarly, there is an expectation with N4820 and P1429 that build levels can be set to other than <code>off</code>, but they do allow global control to do that (so at least one potential program behavior would be buildable without needing to make contract annotations do).</p> <p>Obviously a macro-based solution would be able to provide this kind of feature.</p>                                                                                                                                       |                                               |                       |                                      |                                                                                                                                               |    |    |    |
| 31                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">ccppdev.interop</a>               | Mixed C/C++ Developer | Maintain mixed code base             | Not lose contracts when crossing languages                                                                                                    | 0  | 0  | 0  |
| 32                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">cdev.cppinterop</a>               | Mixed C/C++ Developer | Write contracts on my functions      | Expose my contracts to C++ developers through 'extern "C"' declarations of my functions                                                       | 0  | 0  | 0  |
| A pure macro-based facility exposes no contracts to callers, regardless of what language the callers are calling from. The language-based facilities would not have readily standardized by WG14.                                                                                                                                                                                                                                                                                                                                                                                                           |                                               |                       |                                      |                                                                                                                                               |    |    |    |
| 33                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <a href="#">api.communicate.inputsoutputs</a> | API Developer         | Communicate my interface to          | Document the expected inputs and expected outputs on my interface                                                                             | 25 | 75 | 75 |

|    |                                              |               |                      |                                                                          |    |    |    |
|----|----------------------------------------------|---------------|----------------------|--------------------------------------------------------------------------|----|----|----|
|    |                                              |               | users                |                                                                          |    |    |    |
| 34 | <a href="#">api.establish.check</a>          | API Developer | Establish a contract | Have validation inform me which output values are unexpected or invalid  | 25 | 75 | 75 |
| 35 | <a href="#">api.establish.values</a>         | API Developer | Establish a contract | Have validation inform user which input values are unexpected or invalid | 25 | 75 | 75 |
| 36 | <a href="#">api.establish.preconditions</a>  | API Developer | Establish a contract | Have contracts specify their pre-conditions as logical predicates        | 25 | 75 | 75 |
| 37 | <a href="#">api.establish.postconditions</a> | API Developer | Establish a contract | Have contracts specify their post-conditions as logical predicates       | 25 | 75 | 75 |

Precondition and postcondition checks within a function do something to document inputs and outputs, as do comments that are readily available without a language feature.

The language proposals all included the common feature of `[[pre]]` and `[[post]]` to express this information directly on function declarations, and have those get checked at runtime.

|    |                                                   |                   |                      |                                                                           |   |   |   |
|----|---------------------------------------------------|-------------------|----------------------|---------------------------------------------------------------------------|---|---|---|
| 38 | <a href="#">api.establish.validate_invariants</a> | API Developer     | Establish a contract | Have validation inform me which class invariants are violated             | 5 | 5 | 5 |
| 39 | <a href="#">api.establish.invariants</a>          | API Developer     | Establish a contract | Have contracts specify their class invariants as logical predicates       | 5 | 5 | 5 |
| 40 | <a href="#">cppapi.invariants</a>                 | C++ API Developer | Write classes        | Declare class invariants that all of my public functions need to maintain | 0 | 0 | 0 |

Neither a macro based facility or the language proposals allowed for specifying at a class level invariants that would be automatically checked (barring manually adding class as pre and post conditions on every single function.)

|    |                                              |               |                      |                                                                                                                                                                  |    |    |    |
|----|----------------------------------------------|---------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|
| 41 | <a href="#">api.express.values</a>           | API Developer | Express predicates   | Make reference to either the values of my inputs, or other in-scope identifiers                                                                                  | 75 | 75 | 75 |
| 42 | <a href="#">api.establish.changedvalues</a>  | API Developer | Establish a contract | Make reference to the before and after values of in-out variables (ie. passed by pointer or reference) in post-conditions                                        | 40 | 0  | 0  |
| 43 | <a href="#">api.establish.changedmembers</a> | API Developer | Establish a contract | Make reference to the before and after values of mutable class members (eg. <code>new_size = old_size+1</code> after <code>push_back</code> ) in post-conditions | 40 | 0  | 0  |
| 44 | <a href="#">api.establish.changedstate</a>   | API Developer | Establish a contract | Make reference to the before and after values of global state (eg., <code>global &gt;= old(global) + 1</code> ) in post-conditions                               | 40 | 0  | 0  |

A macro based facility can similarly expose macros to control the enablement of arbitrary code at arbitrary scopes, and so enables storing values between calling time and return time manually.

The language-based facilities allow for referencing inputs and outputs, with the caveat that inputs were only referencable in outputs if they were not changed. The language facilities explicitly prevented any form of making additional code execute if contracts were enabled, and thus removed the ability to store data between the start and end of a function.

|    |                                            |               |                            |                                                                                   |   |   |   |
|----|--------------------------------------------|---------------|----------------------------|-----------------------------------------------------------------------------------|---|---|---|
| 45 | <a href="#">api.extend.exceptionsafety</a> | API Developer | Extend contractual aspects | Annotate operations as being exception safe                                       | 0 | 0 | 0 |
| 46 | <a href="#">api.extend.threadsafety</a>    | API Developer | Extend contractual aspects | Annotate operations as being thread safe                                          | 0 | 0 | 0 |
| 47 | <a href="#">api.extend.atomicity</a>       | API Developer | Extend contractual aspects | Annotate operations as being atomic (ie. all or no changes become visible)        | 0 | 0 | 0 |
| 48 | <a href="#">api.extend.realtime</a>        | API Developer | Extend contractual aspects | Annotate operations as real-time (ie. guaranteed to complete within a time frame) | 0 | 0 | 0 |
| 49 | <a href="#">api.extend.determinism</a>     | API Developer | Extend contractual aspects | Annotate operations as being deterministic (ie. same outputs for same inputs)     | 0 | 0 | 0 |

|    |                                        |               |                            |                                                                                                        |   |   |   |
|----|----------------------------------------|---------------|----------------------------|--------------------------------------------------------------------------------------------------------|---|---|---|
| 50 | <a href="#">api.extend.purity</a>      | API Developer | Extend contractual aspects | Annotate operations as functionally pure (ie. no side effects)                                         | 0 | 0 | 0 |
| 51 | <a href="#">api.extend.sideeffects</a> | API Developer | Extend contractual aspects | Annotate operations as having global side effects (ie. write to singleton, file, network, or database) | 0 | 0 | 0 |
| 52 | <a href="#">api.extend.complexity</a>  | API Developer | Extend contractual aspects | Annotate algorithmic complexity                                                                        | 0 | 0 | 0 |

No codification of these features is baked into the language itself or added by the previous proposals.

|    |                                             |               |                               |                                                                                                                              |   |   |    |
|----|---------------------------------------------|---------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------|---|---|----|
| 53 | <a href="#">api.express.runnability</a>     | API Developer | Express unrunnable contracts  | Be able to use a predicate that is not evaluated at runtime, because it might be unsafe to run or have stateful side effects | 0 | 0 | 75 |
| 54 | <a href="#">api.express.undefined</a>       | API Developer | Express unrunnable contracts  | Be able to use a predicate that doesn't have a definition, because it hasn't been written yet, or is infeasible to run       | 0 | 0 | 75 |
| 55 | <a href="#">api.express.uncheckable</a>     | API Developer | Express uncheckable contracts | Be able to use a predicate that is not evaluated, because it is simply a semantic placeholder for a tool                     | 0 | 0 | 75 |
| 56 | <a href="#">api.express.unimplementable</a> | API Developer | Express uncheckable contracts | Be able to use a predicate that cannot have a complete definition, because it is inexpressible in the language               | 0 | 0 | 75 |

There is no current in-language way to express predicates that will not be executed at runtime.

N4820 had unchecked contracts leave their evaluation as unspecified, thus making any predicate that has UB or has side effects potentially evaluated (specifically, the word "evaluated" is used to mean "evaluated and not evaluated".)

P1429 and P1607's assume semantic, however, attempted to capture the original intent of the contract proposals and make unchecked contracts not be evaluated, thus ensuring that predicates that are not themselves safe to ever evaluate.

|    |                                              |               |                                     |                                                                                                                                                                                                            |   |    |    |
|----|----------------------------------------------|---------------|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|
| 57 | <a href="#">api.establish.responsibility</a> | API Developer | Establish responsibility boundaries | Inform users which errors are the responsibility of the caller, and which are the callee                                                                                                                   | 0 | 50 | 50 |
| 58 | <a href="#">api.resp.preassert</a>           | API Developer | Establish responsibility boundaries | Annotate assertions inside function bodies that indirectly test preconditions (such as malformed data discovered while performing the algorithm) should be reported to the caller as precondition failures | 0 | 0  | 0  |

Macros within a function are unable to readily gather information about what called the function, thus making it infeasible to identify calling code in a precondition violation in macro-based facility.

N4820 made it possible to change that for `[[pre]]` annotations on a function declaration, allowing for the reported source line to be the calling location if the implementation is able to do so. There was, however, still no facility to explicitly point at a caller being the source of a problem when the problem gets detected within the body of a function instance, when delegating to another function without duplicating that function's preconditions explicitly).

|    |                                        |               |                                       |                                              |   |     |     |
|----|----------------------------------------|---------------|---------------------------------------|----------------------------------------------|---|-----|-----|
| 59 | <a href="#">api.contract.interface</a> | API Developer | Have contract as part of my interface | Declare contract when I declare the function | 0 | 100 | 100 |
|----|----------------------------------------|---------------|---------------------------------------|----------------------------------------------|---|-----|-----|

This is not doable with macros, but was inherent in the proposed contract solutions.

|    |                                      |               |                                          |                                                                                                                               |     |     |     |
|----|--------------------------------------|---------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-----|-----|-----|
| 60 | <a href="#">api.contract.private</a> | API Developer | Keep my user interfaces clean and narrow | Be able to access private implementation details of the class so I don't have to widen public interface to declare predicates | 100 | 100 | 100 |
|----|--------------------------------------|---------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-----|-----|-----|

A macro within a function has full access to private data members of that function's class, as do contracts in all of the language contract proposals.

|    |                                            |               |                                             |                                                                                                                                                                                               |   |   |   |
|----|--------------------------------------------|---------------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|
| 61 | <a href="#">api.contract.redeclaration</a> | API Developer | Keep my public interfaces clean and concise | Place function contract conditions on any declaration (e.g., on redeclarations at the bottom of the header, or on the definition in an implementation file, where they are less distracting). | 0 | 0 | 0 |
|----|--------------------------------------------|---------------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|

A macro-based facility obviously has no contracts on any declarations. The language based proposals did not allow for contracts on any declaration except the first one.

Note that [P1320R1](#) was going to be presented in cologne and sought to alter this state.

|    |                                            |               |                                               |                                                                                                                                                                       |    |    |    |
|----|--------------------------------------------|---------------|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|
| 62 | <a href="#">api.contract.errorhandling</a> | API Developer | Move contract violation out of error handling | Replace uses of error handling to express contract violation (eg. <code>operator[](size_t n)</code> <code>noexcept [[pre: n &lt; size()]]</code> instead of throwing) | 50 | 50 | 50 |
|----|--------------------------------------------|---------------|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|

This is a design decision when defining an API that could be readily accomplished with any contract facility.

|    |                                             |                   |                             |                                                                                                                                                   |   |    |    |
|----|---------------------------------------------|-------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|
| 63 | <a href="#">cppapi.class.preconditions</a>  | C++ API Developer | Maintain a class hierarchy  | Ensure overriding methods have same or wider preconditions (see: Liskov substitution principle)                                                   | 0 | 40 | 40 |
| 64 | <a href="#">cppapi.class.postconditions</a> | C++ API Developer | Maintain a class hierarchy  | Ensure overriding functions meet their base class postconditions when their base class preconditions are met (see: Liskov substitution principle) | 0 | 40 | 40 |
| 65 | <a href="#">cppapi.class.variability</a>    | C++ API Developer | Maintain a class hierarchy. | Allow overriding functions to have narrower preconditions/wider postconditions if I want to                                                       | 0 | 0  | 0  |

A wholly macro based facility has no way to automatically integrate with function overrides.

The language proposals all required that virtual functions have exactly the same preconditions and postconditions, and so allowed for a subset of what would be Liskov sub

|    |                                             |                   |                                  |                                                                                                                                                                                                      |   |   |   |
|----|---------------------------------------------|-------------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|
| 66 | <a href="#">api.class.publicinterface</a>   | C++ API Developer | Express public class invariants  | Express a restriction on the public interface of a type that all callers of the type can depend upon: can mention only public members, and is checked on entry and exit from this type's code        | 0 | 0 | 0 |
| 67 | <a href="#">api.class.publicinvariants</a>  | C++ API Developer | Express public class invariants  | Check invariants before and after every public method (when called from outside the type, not when one member function calls another)                                                                | 0 | 0 | 0 |
| 68 | <a href="#">api.class.publiccalls</a>       | C++ API Developer | Express public class invariants  | Check invariants before and after calling functions that are not part of this type (including virtual calls)                                                                                         | 0 | 0 | 0 |
| 69 | <a href="#">api.class.baseinterface</a>     | C++ API Developer | Express base class invariants    | Express a restriction on the protected interface of a type that derived types can depend upon: can mention only protected and public members, and is checked on entry and exit from this type's code | 0 | 0 | 0 |
| 70 | <a href="#">api.class.baseinvariants</a>    | C++ API Developer | Express base class invariants    | Check invariants on entry and exit of every protected method (when called from the derived type, not when one base member function calls another)                                                    | 0 | 0 | 0 |
| 71 | <a href="#">api.class.basecalls</a>         | C++ API Developer | Express base class invariants    | Check invariants before and after every call to a virtual function (when calling to the derived type)                                                                                                | 0 | 0 | 0 |
| 72 | <a href="#">api.class.privateinterface</a>  | C++ API Developer | Express private class invariants | Express an internal restriction on the private implementation of a type, can mention any member, and is checked on entry and exit from this type's code                                              | 0 | 0 | 0 |
| 73 | <a href="#">api.class.privateinvariants</a> | C++ API Developer | Express private class invariants | Check invariants on entry and exit of every public method (when called from outside the type, not when one member function calls another)                                                            | 0 | 0 | 0 |
| 74 | <a href="#">api.class.privatecalls</a>      | C++ API Developer | Express private class            | Check invariants before and after calling functions that are not part of this type (including                                                                                                        | 0 | 0 | 0 |



|                                                                                                                                                                                                                                                                                          |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-----------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|-----|
|                                                                                                                                                                                                                                                                                          |                                            |                       | invariants                               | virtual calls)                                                                                                                                                                                                                                                                            |     |     |     |
| A macro-based facility is not part of the declared interface to any class, but only part of the implementation.                                                                                                                                                                          |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| The language proposals were limited to functions but not types, and always had private access.                                                                                                                                                                                           |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| 75                                                                                                                                                                                                                                                                                       | <a href="#">api.class.testing</a>          | C++ API Developer     | Test my classes                          | For every member or friend function in my class, run my unit test framework with checking enabled for every assertion at the point where it is written, and check every postcondition at every non-exceptional exit, and test my class invariants on entry and exit from this type's code | 0   | 0   | 0   |
| This level of control is not providable for macros or the language proposals.                                                                                                                                                                                                            |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| 76                                                                                                                                                                                                                                                                                       | <a href="#">cppapi.contracts.async</a>     | C++ API Developer     | Enforce contracts in async code          | Express contracts on callbacks such as <code>std::function</code> , function pointers, or references to functions, lambdas, or function objects                                                                                                                                           | 0   | 0   | 0   |
| This would require contracts be part of a function type. Macros are not even part of a function declaration, let alone its type, and the previous language proposals did not check contracts part of a function type either.                                                             |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| 77                                                                                                                                                                                                                                                                                       | <a href="#">cppapi.contracts.exception</a> | C++ API Developer     | Enforce contracts in exception safe code | Express contracts on exceptional exit                                                                                                                                                                                                                                                     | 0   | 0   | 0   |
| 78                                                                                                                                                                                                                                                                                       | <a href="#">cppapi.variadic</a>            | C++ API Developer     | Use contracts with variadic templates    | Allow predicate (fold) expansion                                                                                                                                                                                                                                                          | 0   | 0   | 0   |
| Neither of these are viable on the.                                                                                                                                                                                                                                                      |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| 79                                                                                                                                                                                                                                                                                       | <a href="#">int.conform.violation</a>      | Integration Developer | Conform to a contract                    | Be informed any time an interface's contract is violated                                                                                                                                                                                                                                  | 25  | 50  | 50  |
| 80                                                                                                                                                                                                                                                                                       | <a href="#">int.conform.postconditions</a> | Integration Developer | Conform to a contract                    | Verify results from a call are expected output values                                                                                                                                                                                                                                     | 25  | 50  | 50  |
| This is the purpose of a checked contract facility, though the granularity of control to limit to or at least include a particular (or all) interface is not there for all proposals.                                                                                                    |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| 81                                                                                                                                                                                                                                                                                       | <a href="#">int.build.headeronly</a>       | Integration Developer | Build multiple libraries                 | Use contract-enabled header-only libraries                                                                                                                                                                                                                                                | 50  | 100 | 100 |
| 82                                                                                                                                                                                                                                                                                       | <a href="#">cpplib.headeronly</a>          | C++ Library Developer | Use templates                            | Be able to ship header only library                                                                                                                                                                                                                                                       | 50  | 100 | 100 |
| A macro based facility is likely to depend on having a supporting library exist to make customization and violation handling available.                                                                                                                                                  |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| Nothing about the proposed solutions required this (or, as a language facility, a library got the supporting functionality needed for free from the compiler's runtime environment)                                                                                                      |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| 83                                                                                                                                                                                                                                                                                       | <a href="#">int.build.binaries</a>         | Integration Developer | Build multiple libraries                 | Use contract-enabled binary libraries                                                                                                                                                                                                                                                     | 100 | 100 | 100 |
| 84                                                                                                                                                                                                                                                                                       | <a href="#">int.build.binarycounts</a>     | Integration Developer | Build multiple libraries                 | Only be required to manage a small, common set of build/link configurations                                                                                                                                                                                                               | 75  | 40  | 25  |
| Policies on how many different builds are managed and deployed are entirely up to users when the facility's controls are all done through macros (either entirely in N4842 or literal semantics in P1607).                                                                               |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| N4820 includes 5 distinct build configurations that might be desired by clients of a library, and P1429 increases that number to 32, though most are only interesting in specific is likely that individual vendors would find a small set that are of actual interest to their clients. |                                            |                       |                                          |                                                                                                                                                                                                                                                                                           |     |     |     |
| 85                                                                                                                                                                                                                                                                                       | <a href="#">int.build.control</a>          | Integration Developer | Debug multiple libraries                 | Enable checks only within a selected library                                                                                                                                                                                                                                              | 50  | 35  | 35  |
| 86                                                                                                                                                                                                                                                                                       | <a href="#">int.build.control2</a>         | Integration Developer | Debug multiple libraries                 | Enable checks on multiple libraries simultaneously                                                                                                                                                                                                                                        | 50  | 35  | 35  |

The build levels are global, with conditional (and potentially no) support for using mixed build levels across different libraries. Macro based solutions are viable as long as a contract always has the same meaning in different translation units (without ODR violations).

|    |                                     |                       |                           |                                           |   |    |    |
|----|-------------------------------------|-----------------------|---------------------------|-------------------------------------------|---|----|----|
| 87 | <a href="#">int.debug.callsites</a> | Integration Developer | Debug multiple call sites | Enable checks only on selected call sites | 0 | 25 | 25 |
|----|-------------------------------------|-----------------------|---------------------------|-------------------------------------------|---|----|----|

None of the solutions allowed for a function to be called with different checking levels without violating the ODR.

|    |                                            |                       |                          |                                                                                                                                            |    |     |     |
|----|--------------------------------------------|-----------------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----|-----|-----|
| 88 | <a href="#">int.violations.information</a> | Integration Developer | Correct failed checks    | Be informed what check failed, when, where, and how                                                                                        | 75 | 100 | 100 |
| 89 | <a href="#">int.violations.transmit</a>    | Integration Developer | Correct failed checks    | Transmit check failure information in environment-specific ways (logs, email, special hardware traps, popup windows, blazing sirens, etc). | 50 | 90  | 90  |
| 90 | <a href="#">int.violations.custom</a>      | Integration Developer | Correct failed checks    | Install custom violation handler where I can inject custom logic to trap errors                                                            | 50 | 90  | 90  |
| 91 | <a href="#">int.violations.common</a>      | Integration Developer | Unify violation handling | Be able to override how library violations are handled in the combined software to point into my handling code                             | 50 | 90  | 90  |

All of the facilities support checking contracts at runtime and being informed of details of the violation. The language proposals included a pluggable violation handler, although vendors were allowed to not make that customizable.

A pure macro-based facility suffers from needing to do this for all of the contract facilities that might be defined and used within a given fully assembled program.

|    |                                         |                       |                                     |                                                                  |    |   |   |
|----|-----------------------------------------|-----------------------|-------------------------------------|------------------------------------------------------------------|----|---|---|
| 92 | <a href="#">int.violations.override</a> | Integration Developer | Be independent of build environment | Be able to define and override violation handler via source code | 15 | 0 | 0 |
|----|-----------------------------------------|-----------------------|-------------------------------------|------------------------------------------------------------------|----|---|---|

This might be doable with a fair bit of complexity in a macro-based facility, and there was no support for this in the language proposals.

|    |                                             |                              |                                            |                                                                                                                                                                 |    |   |   |
|----|---------------------------------------------|------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|---|---|
| 93 | <a href="#">int.build.minimize</a>          | Integration Developer        | Minimize checking overhead                 | Disable library postconditions, asserts, and invariants, without disabling library preconditions (assuming the library is tested and stable and my code is not) | 25 | 0 | 0 |
| 94 | <a href="#">int.control.build</a>           | Integrated Software Provider | Ensure the combined software is correct    | At build time, turn on and off what checking happens.                                                                                                           | 25 | 0 | 0 |
| 95 | <a href="#">int.control.runtime</a>         | Integrated Software Provider | At runtime, control what checking happens. | Turn checks on at run time                                                                                                                                      | 25 | 0 | 0 |
| 96 | <a href="#">int.conrol.subsets.build</a>    | Integrated Software Provider | Ensure the combined software is correct    | Turn on any subset of individual (call site) checks on at build time                                                                                            | 25 | 0 | 0 |
| 97 | <a href="#">int.control.subsets.runtime</a> | Integrated Software Provider | Ensure the combined software is correct    | Turn on any subset of individual (call site) checks on at run time                                                                                              | 25 | 0 | 0 |
| 98 | <a href="#">int.control.subsets</a>         | Integrated Software Provider | Ensure individual features are correct     | Have a way to audit (named or semantic) subsets of checks for various deployments                                                                               | 25 | 0 | 0 |
| 99 | <a href="#">int.testing.control</a>         | Integrated Software Provider | Define "Code Under Test"                   | Selectively enable checking for a set of functions which could name either an individual function or an overload set                                            | 15 | 0 | 0 |

|                                                                                                                                                                                                                                                                                                                                                                             |                                          |                                 |                                                                                              |                                                                                                                                                                                                |    |    |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|---------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|
| 100                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">int.testing.controltypes</a> | Integrated Software Provider    | Define "Code Under Test"                                                                     | Selectively enable checking for a set of types and all their members                                                                                                                           | 15 | 0  | 0  |
| 101                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">int.testing.transitivity</a> | Integrated Software Provider    | Define "Code Under Test"                                                                     | Selectively enable checking for a set of types and all their transitively nested types and members                                                                                             | 5  | 0  | 0  |
| 102                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">int.testing.modules</a>      | Integrated Software Provider    | Define "Code Under Test"                                                                     | Selectively enable checking for a translation unit or module and all (non transitive) types and functions within                                                                               | 5  | 0  | 0  |
| Different forms of build and runtime control could be built into the macro-based facilities with varying levels of difficulty. N4820 and P1429 provided only a global level of control, greatly the ability to control contract enablement at finer granularities. Implementing some of these forms of granularity in the preprocessor might, however, be very complicated. |                                          |                                 |                                                                                              |                                                                                                                                                                                                |    |    |    |
| 103                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">int.consistency</a>          | Integrated Software Provider    | Ensure the combined software is correct                                                      | Verify all annotations are globally consistent when integrated                                                                                                                                 | 0  | 0  | 0  |
| There is no support for this form of checking on a macro-based facility, or from the language proposals.                                                                                                                                                                                                                                                                    |                                          |                                 |                                                                                              |                                                                                                                                                                                                |    |    |    |
| 104                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">int.build.common</a>         | Integrated Software Provider    | Manage binary delivery                                                                       | Be able to use the same executable regardless of contract enforcement mode                                                                                                                     | 15 | 0  | 0  |
| A C++20 based facility could determine checking based on runtime configuration instead of only compile-time configuration. The language proposals did not allow for that.                                                                                                                                                                                                   |                                          |                                 |                                                                                              |                                                                                                                                                                                                |    |    |    |
| 105                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">int.build.unchecked</a>      | Integrated Software Provider    | Test final deliverable                                                                       | Turn off build time checking to remove checking overhead                                                                                                                                       | 40 | 50 | 50 |
| 106                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">int.runtime.unchecked</a>    | Integrated Software Provider    | Test final deliverable                                                                       | Turn off run time checking to remove checking overhead                                                                                                                                         | 50 | 50 | 50 |
| Macros and the language proposals had minimal concepts of "build time checking", but they do all enable a global ability to turn off such checking. That same global ability could also be used to turn off runtime checking.                                                                                                                                               |                                          |                                 |                                                                                              |                                                                                                                                                                                                |    |    |    |
| 107                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">int.build.optimize</a>       | Integrated Software Provider    | Test final deliverable                                                                       | Turn on run time optimization to leverage annotation assumptions                                                                                                                               | 25 | 50 | 75 |
| 108                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">pdev.speed</a>               | Performance Sensitive Developer | Enable better performance                                                                    | Annotate my code with assumptions, likelihoods, or reachability information that a tool might not be able to deduce, but that I would be confident of                                          | 25 | 50 | 75 |
| 109                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">pdev.morespeed</a>           | Performance Sensitive Developer | Enable better performance                                                                    | Be able to give statically-unprovable facts to current and novel optimizers in terms of semantics my program does not depend-on but optimizers can't figure out                                | 25 | 50 | 75 |
| 110                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">pdev.footgun</a>             | Performance Sensitive Developer | Enable better performance                                                                    | Accept responsibility for a malformed program that might result from eventually false information given by my annotations                                                                      | 25 | 50 | 75 |
| A macro based facility would be able to enable optimizations, but that is likely going to have limited direct support from most compilers. (Generally, this would leverage either <code>__unreachable()</code> or <code>__assume()</code> .)                                                                                                                                |                                          |                                 |                                                                                              |                                                                                                                                                                                                |    |    |    |
| N4820 made optimizations enabled for ALL unchecked contracts, while P1429 and P1607 gave mechanisms to opt into that in varying ways.                                                                                                                                                                                                                                       |                                          |                                 |                                                                                              |                                                                                                                                                                                                |    |    |    |
| 111                                                                                                                                                                                                                                                                                                                                                                         | <a href="#">cpplib.insulation</a>        | C++ Library Developer           | Control the tradeoff between need for client recompilation and contract condition visibility | Insulate contract conditions with the function definition, or insulate only the definition while putting contract conditions on a redeclaration - visible to static analysis tools in all TUs. | 25 | 50 | 50 |

Macro based contract checks will always be in the function body and thus fully insulated from clients.

The contract annotations of the language proposals could be equivalently put in function bodies with `[[assert]]` or on the declarations, thus giving some control over ability to have them visible to clients through a redeclaration but not visible on the initial declaration was not, however, in the initial proposals (but was proposed but not seen [P1320R1](#).)

|     |                                          |                  |                                |                                                          |    |    |    |
|-----|------------------------------------------|------------------|--------------------------------|----------------------------------------------------------|----|----|----|
| 112 | <a href="#">lib.maintenance.noconfig</a> | Library Provider | Simplify maintenance           | Not require extra build steps to be documented           | 25 | 25 | 25 |
| 113 | <a href="#">lib.integration.noconfig</a> | Library Provider | Support successful integration | Not require extra build steps to be learned or performed | 25 | 25 | 25 |

Any contract facility that allows control without source code manipulation will require some build time control, either through compiler flags or macros, though all should have with some default behavior if no explicit choices were made at build time.

|     |                                           |                  |                                |                                                                                                                       |    |   |   |
|-----|-------------------------------------------|------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------|----|---|---|
| 114 | <a href="#">lib.maintenance.nowhining</a> | Library Provider | Simplify maintenance           | Not have users complain about my product due to modifications of annotations resulting from their build configuration | 75 | 0 | 0 |
| 115 | <a href="#">lib.integration.nowhining</a> | Library Provider | Support successful integration | Not have my users accidentally modify my careful annotations                                                          | 75 | 0 | 0 |

A macro based facility could provide concrete semantics similar to P1607, thus making contracts that cannot have their behavior changed from the command line.

N4820 and P1429 contracts are always subject to build modes, leaving no way to enforce that a particular behavior is applied to a given contract annotation.

|     |                               |                     |                               |                                                                                                                                                                                                                                                            |   |    |    |
|-----|-------------------------------|---------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|
| 116 | <a href="#">arch.nomacros</a> | Technical Architect | Maintain quality of code base | Express assertions in a way that does not rely on C macros (i.e., there is no valid technical reason for a programmer not to use the new way, including space, time, tooling, and usability/complexity reasons, compared to C's <code>assert</code> macro) | 0 | 50 | 50 |
|-----|-------------------------------|---------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|

Contract checks without a language facility are not feasibly doable without macros.

All of the language proposals require macros in some form or other to satisfy many of the use cases in this document, but still provide a basic contract checking facility with used at all.

|     |                               |                     |                                                   |                                                                                                                                                                                                                                                                                                                |   |    |    |
|-----|-------------------------------|---------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|
| 117 | <a href="#">arch.complete</a> | Technical Architect | Have a consistent and holistic contracts facility | Specify preconditions/postconditions/assertions/invariants that express my expectations about the expected valid state of my program in the form of compilable boolean expressions, that can be checked statically or dynamically (as opposed to disjointed state where these features are factored into bits) | 0 | 75 | 75 |
|-----|-------------------------------|---------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|

Obviously the language does not have this now, and the language proposals all attempted to provide this.

|     |                                      |                    |                                  |                                                                                                                                                               |   |    |    |
|-----|--------------------------------------|--------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|
| 118 | <a href="#">hardware.performance</a> | Hardware Architect | Improve system-level performance | Be able to design new hardware + optimizations, carefully dovetailed into one another, that depend on statically-unprovable facts being annotated in the code | 0 | 25 | 75 |
|-----|--------------------------------------|--------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|

N4820's inability to have contracts that are never executed prevents this from being leveraged for many novel features. P1429 and P1607's assume semantic fixed this.

|     |                                    |                  |                |                                                    |    |    |    |
|-----|------------------------------------|------------------|----------------|----------------------------------------------------|----|----|----|
| 119 | <a href="#">sdev.bestpractices</a> | Senior Developer | Set an example | Demonstrate best practice in defensive programming | 50 | 25 | 50 |
|-----|------------------------------------|------------------|----------------|----------------------------------------------------|----|----|----|

Any basic contract checking facility can be used to implement defensive programming.

N4820's implicit assumption of any unchecked contract, however, is unlikely to ever be viewable as a best practice to use by anyone.

|     |                              |                  |                      |                                                                                                                   |    |    |    |
|-----|------------------------------|------------------|----------------------|-------------------------------------------------------------------------------------------------------------------|----|----|----|
| 120 | <a href="#">sdev.quality</a> | Senior Developer | Enforce code quality | Discourage reliance on observable out-of-contract behavior by causing check failure to hard stop program or build | 50 | 50 | 50 |
|-----|------------------------------|------------------|----------------------|-------------------------------------------------------------------------------------------------------------------|----|----|----|

This behavior is available in any of the facilities, though not as strongly enforced when a user has control over the violation handler.

|     |                               |                  |                                     |                                                                              |    |    |    |
|-----|-------------------------------|------------------|-------------------------------------|------------------------------------------------------------------------------|----|----|----|
| 121 | <a href="#">sdev.maturity</a> | Senior Developer | Enforce mature, finalized contracts | Disable continuation on violation of stable and correct individual contracts | 50 | 25 | 25 |
| 122 | <a href="#">sdev.control</a>  | Senior Developer | Enforce mature, finalized contracts | Disable remapping of semantics on stable and correct individual contracts    | 50 | 25 | 25 |

The global controls of N4820 and P1429 do not allow for control over an individual contract's behavior (continuing or not, checked or not) based on the maturity of that spec

|     |                                                |                  |                        |                                                                                                                                       |    |    |    |
|-----|------------------------------------------------|------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------|----|----|----|
| 123 | <a href="#">jdev.understand.contracts</a>      | Junior Developer | Understand the API     | A uniform, fluent description of expected input values, expected output values, side effects, and all logical pre and post conditions | 0  | 75 | 75 |
| 124 | <a href="#">jdev.understand.violations</a>     | Junior Developer | Understand the API     | Be informed when my usage is out of contract                                                                                          | 50 | 75 | 75 |
| 125 | <a href="#">jdev.understand.buildfailures</a>  | Junior Developer | Understand the program | Know why my software is not building                                                                                                  | 50 | 75 | 75 |
| 126 | <a href="#">jdev.understand.aborting</a>       | Junior Developer | Understand the program | Know why my software is aborting                                                                                                      | 50 | 75 | 75 |
| 127 | <a href="#">jdev.understand.omniscience</a>    | Junior Developer | Understand the program | Know why my software is out of contract                                                                                               | 50 | 75 | 75 |
| 128 | <a href="#">jdev.understand.buildviolation</a> | Junior Developer | Understand the program | Know that my program or build was halted due to contract violation                                                                    | 50 | 75 | 75 |
| 129 | <a href="#">jdev.bestpractices</a>             | Junior Developer | Improve my code        | Learn about software best practices by example                                                                                        | 50 | 75 | 75 |

Without a language feature contract descriptions are library-specific and not uniform. With it, the only non-uniformity comes in when libraries build extra infrastructure on top language-provided facility.

A library and a language-based facility will, however, be able to provide a user understandable details of why a contract violation might have made a program abort or fail to

|     |                                     |                  |                         |                                                                           |    |    |    |
|-----|-------------------------------------|------------------|-------------------------|---------------------------------------------------------------------------|----|----|----|
| 130 | <a href="#">jdev.understand.all</a> | Junior Developer | Understand the facility | Be able to build a program with contracts after reasonably short tutorial | 50 | 75 | 75 |
|-----|-------------------------------------|------------------|-------------------------|---------------------------------------------------------------------------|----|----|----|

Whether with a library or any of the proposed language features, simple contract use remains simple.

|     |                                          |                  |                         |                                                     |   |    |    |
|-----|------------------------------------------|------------------|-------------------------|-----------------------------------------------------|---|----|----|
| 131 | <a href="#">jdev.understand.keywords</a> | Junior Developer | Understand the facility | Have keywords with precise and unambiguous meanings | 0 | 25 | 25 |
|-----|------------------------------------------|------------------|-------------------------|-----------------------------------------------------|---|----|----|

Without a language feature there are no new keywords. N4820 and P1429 include `axiom` which has shown to have significant disagreement over what meaning it has in contracts (see [P1672](#)).

The keywords provided by P1607 are all very precisely defined.

|     |                           |                 |                 |                                                                                           |   |    |    |
|-----|---------------------------|-----------------|-----------------|-------------------------------------------------------------------------------------------|---|----|----|
| 132 | <a href="#">adev.fast</a> | Agile Developer | Iterate quickly | Be able to write and modify contracts quickly without heavy boiler plate or up front cost | 0 | 75 | 75 |
|-----|---------------------------|-----------------|-----------------|-------------------------------------------------------------------------------------------|---|----|----|

The general use of contracts once they are a language feature is quick to get started on. Without a language feature, getting started requires acquiring or implementing a lib provide the feature.

|     |                             |                 |                         |                                                                         |    |   |   |
|-----|-----------------------------|-----------------|-------------------------|-------------------------------------------------------------------------|----|---|---|
| 133 | <a href="#">adev.evolve</a> | Agile Developer | Safeguard evolving code | Assert against conditions I am aware of but not finished handling fully | 20 | 0 | 0 |
|-----|-----------------------------|-----------------|-------------------------|-------------------------------------------------------------------------|----|---|---|

Without a language-based facility the only way to enter a not-yet-implemented contract is as a comment, which is of limited utility.

P1607's ignore and assume semantics both allow for referencing undefined functions, and thus enable writing a planned contract while preserving writing the implementation check for a future sprint. N4820 provides no way to get that semantic for a contract annotation, and while P1429's assume semantic would, there is no way in code to write : contract.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                             |                                 |                                |                                                                                                                                                                                      |    |     |     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|---------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|-----|-----|
| 134                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">bdev.confidentiality</a>        | Business Developer              | Maintain confidentiality       | Not expose diagnostic information (source location, expressions, etc.) in the software I deliver to clients, even when I choose to have contracts enforced in the software I deliver | 25 | 0   | 0   |
| <p>A macro-based library could choose to forgo including this information, or provide flags to control that.</p> <p>None of the language features proposed include that ability.</p>                                                                                                                                                                                                                                                           |                                             |                                 |                                |                                                                                                                                                                                      |    |     |     |
| 135                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">pdev.safety.isolation</a>       | Performance Sensitive Developer | Have safety critical paths     | Isolate safety checks from performance annotations                                                                                                                                   | 25 | 0   | 0   |
| <p>A macro-based library could provide this distinction. None of the language proposals include this, and importantly all of them allow for a contract to be assumed and thus su future "safety checks" that attempt to handle out-of-contract behavior more elegantly.</p>                                                                                                                                                                    |                                             |                                 |                                |                                                                                                                                                                                      |    |     |     |
| 136                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">pdev.safety.critical</a>        | Performance Sensitive Developer | Have safety critical paths     | Retain checking even when optimizing with performance annotations                                                                                                                    | 25 | 0   | 15  |
| <p>N4820 provides no way to turn checking off locally without bringing in assumption. P1429 at least provides a way to build an application without assumption of checks, and I solutions on top of P1607 or a fully-macro based solution would be able to allow for this kind of distinction with some effort.</p>                                                                                                                            |                                             |                                 |                                |                                                                                                                                                                                      |    |     |     |
| 137                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">qdev.checkall</a>               | Quality Sensitive Developer     | Enable full checking           | Ensure all checks (pre, post, assert, invariant) are enabled                                                                                                                         | 25 | 100 | 100 |
| 138                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">qdev.fuzz.testing</a>           | Quality Sensitive Developer     | Catch unexpected failure modes | Log all predicate failure during fuzz testing                                                                                                                                        | 25 | 100 | 100 |
| <p>A disparate set of macro-based libraries makes it hard to turn on all checking.</p> <p>N4820 and P1429 allow for this to be done very easily with build modes. P1607 allows for code to subvert this kind of control by providing explicit semantics for a particular cannot be externally altered.</p>                                                                                                                                     |                                             |                                 |                                |                                                                                                                                                                                      |    |     |     |
| 139                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">qdev.correctness</a>            | Quality Sensitive Developer     | Validate correctness           | Signify the predicates that should be verified by an analysis tool                                                                                                                   | 25 | 50  | 50  |
| 140                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">qdev.tooling</a>                | Quality Sensitive Developer     | Manage multiple tools          | Signify subset of individual annotations to be consumed by a specific kind of verification tool                                                                                      | 25 | 50  | 50  |
| 141                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">qdev.tooling.control</a>        | Quality Sensitive Developer     | Manage multiple tools          | Signify subset of individual annotations to be consumed by a specific instance of verification tool                                                                                  | 25 | 0   | 0   |
| <p>It can be argued that the primary intention of <code>axiom</code> was to only provide information to static analysis tools, so both of the proposals that include that have that here, ever wording itself added in undefined behavior to that level, and made no mention of this purpose.</p> <p>A macro-based facility could integrate with a static analysis tool for this purpose, but would require tools to choose to support it.</p> |                                             |                                 |                                |                                                                                                                                                                                      |    |     |     |
| 142                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">qdev.tooling.undefined</a>      | Quality Sensitive Developer     | Manage multiple tools          | Use predicates that may not be understood by all instances of verification                                                                                                           | 0  | 0   | 25  |
| 143                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">qdev.tooling.undefinedkinds</a> | Quality Sensitive Developer     | Manage multiple tools          | Use predicates that may not be understood by all kinds of verification                                                                                                               | 0  | 0   | 25  |
| <p>Only P1429 and P1607 allow predicates to remain undefined if not referenced and not checked at runtime.</p>                                                                                                                                                                                                                                                                                                                                 |                                             |                                 |                                |                                                                                                                                                                                      |    |     |     |
| 144                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">qdev.tooling.behavior</a>       | Quality Sensitive Developer     | Manage multiple tools          | Integrate the results of that static checker into how my program behaves in different ways: assume proven predicates, make unprovable predicates ill- formed, etc.                   | 25 | 0   | 0   |
| <p>P1607 or a wholly macro-based facility provide the only direct way to integrate the results of analysis into specific behaviors for contract checks.</p>                                                                                                                                                                                                                                                                                    |                                             |                                 |                                |                                                                                                                                                                                      |    |     |     |



|                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                          |                              |                                        |                                                                                                                                                                                                                                                                                                                                                                                           |    |    |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|------------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|
| 154                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">crit.production.checking</a> | Critical Software Developer  | Have redundant layering                | Be able to continue to run checks in a production environment (even after formal testing is complete)                                                                                                                                                                                                                                                                                     | 25 | 50 | 75 |
| P1429 gives complete flexibility about what levels are checked or not in the builds you choose to deploy to production. P1607 and a macro-based facility allow building that functionality.                                                                                                                                                                                                                                                        |                                          |                              |                                        |                                                                                                                                                                                                                                                                                                                                                                                           |    |    |    |
| 155                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">crit.more.coverage</a>       | Critical Software Developer  | Maximize coverage                      | Be able to run checks in a production environment that are considered "cheap" compared to the expected cost of entering an invalid state                                                                                                                                                                                                                                                  | 25 | 75 | 75 |
| N4820 and P1429 both explicitly consider cost as the primary metadata that can be put on a contract annotation (via a level of <code>default</code> or <code>audit</code> ). This doesn't allow granularity in that expression of cost, but it is                                                                                                                                                                                                  |                                          |                              |                                        |                                                                                                                                                                                                                                                                                                                                                                                           |    |    |    |
| 156                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">crit.noassume</a>            | Critical Software Developer  | Avoid unexpected or undefined behavior | Ensure checks will never be <code>__assume'd/__/builtin_assume'd</code> by the compiler as if they were facts injected into the program (otherwise, if such an assumption ever failed, I would be running a different program that is not equivalent to the one I wrote; assumptions can expand the set of possible executions by injecting facts not otherwise knowable to the compiler) | 0  | 0  | 50 |
| Only P1429 provides the a way to configure contract annotations to never be assumed.                                                                                                                                                                                                                                                                                                                                                               |                                          |                              |                                        |                                                                                                                                                                                                                                                                                                                                                                                           |    |    |    |
| 157                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">sec.noattacks</a>            | Security Sensitive Developer | Limit attack vectors                   | Be unable to insert code paths (eg. violation handlers) at run time (eg. build time only)                                                                                                                                                                                                                                                                                                 | 0  | 90 | 90 |
| None of the contract facilities allow any runtime alteration of the violation handler.<br><br>Note importantly that an application may choose to install a custom violation handler that delegates to something that is runtime controllable, so this restriction is circumvented by the compiler allows for setting a custom violation handler.                                                                                                   |                                          |                              |                                        |                                                                                                                                                                                                                                                                                                                                                                                           |    |    |    |
| 158                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">sec.certify</a>              | Security Sensitive Developer | Deliver a certified product            | Have build tool only link to a preapproved violation handler                                                                                                                                                                                                                                                                                                                              | 0  | 90 | 90 |
| The wording for N4820 and its derivatives allows for a compiler to choose to not allow for changing the violation handler to something user defined.                                                                                                                                                                                                                                                                                               |                                          |                              |                                        |                                                                                                                                                                                                                                                                                                                                                                                           |    |    |    |
| 159                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">analysis.runtime</a>         | User of Analysis Tools       | Improve runtime correctness            | Have runtime checks generated by the tool                                                                                                                                                                                                                                                                                                                                                 | 0  | 0  | 0  |
| 160                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">analysis.optimization</a>    | User of Analysis Tools       | Improve runtime performance            | Have runtime optimizations generated by the tool                                                                                                                                                                                                                                                                                                                                          | 0  | 0  | 0  |
| None of the proposals or the language itself support direct integration with a static analysis tool. Explicit literal semantics, however, could be injected into code by such a tool to compute contract behavior - runtime checking, optimizations, or otherwise.                                                                                                                                                                                 |                                          |                              |                                        |                                                                                                                                                                                                                                                                                                                                                                                           |    |    |    |
| 161                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">analysis.symbolic</a>        | User of Analysis Tools       | Allow symbolic analysis                | Have symbolic proofs for soundness and consistency performed before compile time                                                                                                                                                                                                                                                                                                          | 25 | 50 | 75 |
| 162                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">analysis.compiletime</a>     | User of Analysis Tools       | Allow code analysis                    | Have code source, AST, or instruction inspection during compile time                                                                                                                                                                                                                                                                                                                      | 25 | 50 | 75 |
| Proofs of soundness are certainly aided by stating contract annotations and asking for validation of those annotations. Tooling will need to catch up to leverage this and do so.<br><br>Many such proofs rely on being able to state additional facts that are not easy to codify as boolean checks, and those are often needed to thoroughly prove even much simple predicates, so none of the proposed solutions are complete for this purpose. |                                          |                              |                                        |                                                                                                                                                                                                                                                                                                                                                                                           |    |    |    |
| 163                                                                                                                                                                                                                                                                                                                                                                                                                                                | <a href="#">analysis.binaries</a>        | User of Analysis Tools       | Allow binary analysis                  | Have binary inspection after compile time                                                                                                                                                                                                                                                                                                                                                 | 0  | 50 | 50 |



In principle contract checks could be carried forward and recorded in binaries to allow for post-compile verification. It seems unlikely that a non-builtin facility would be able to record contract checks in binary files in such a way.

|     |                                      |                        |                                 |                                                                                                                                                    |   |   |   |
|-----|--------------------------------------|------------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|
| 164 | <a href="#">analysis.information</a> | User of Analysis Tools | Improve the quality of analysis | Be able to hint to the analyzer information it may be unable to deduce from source code alone (eg. <code>5 / opaque(); [[ opaque() != 0]]</code> ) | 0 | 0 | 0 |
|-----|--------------------------------------|------------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|

None of the proposals allow for hints that are exclusively for the static analyzer to use, but the P1607 `ignore` semantic could be used for such a purpose.

|     |                                 |                            |                           |                                                                                          |   |    |    |
|-----|---------------------------------|----------------------------|---------------------------|------------------------------------------------------------------------------------------|---|----|----|
| 165 | <a href="#">analysis.legacy</a> | Provider of Analysis Tools | Extend my existing engine | Be able to map pre-existing contract features in tools to a standardized language syntax | 0 | 25 | 25 |
| 166 | <a href="#">large.modernize</a> | Large Codebase Owner       | Modernize my code base    | Introduce standardized contracts to replace my macro-based contracts                     | 0 | 25 | 25 |

Only a very limited legacy framework (such as the C `assert` macro) that completely falls within the functionality provided by N4820 would be migratable without drastic changes in behavior.

P1607 sought to provide more flexibility for reimplementing most legacy frameworks in terms of common semantics provided by the language.

|     |                                     |         |                           |                                                                                                                         |    |     |     |
|-----|-------------------------------------|---------|---------------------------|-------------------------------------------------------------------------------------------------------------------------|----|-----|-----|
| 167 | <a href="#">teach.bestpractices</a> | Teacher | Demonstrate best practice | Be able to express defensive programming, programming by contract, and test driven development to introductory students | 25 | 100 | 100 |
| 168 | <a href="#">teach.standardized</a>  | Teacher | Demonstrate best practice | Not rely on custom libraries or proprietary extensions                                                                  | 0  | 100 | 100 |
| 169 | <a href="#">teach.lifecycle</a>     | Teacher | Demonstrate best practice | Demonstrate mock lifecycle by switching simple compiler flags to control which checks are enabled                       | 0  | 50  | 50  |
| 170 | <a href="#">teach.portable</a>      | Teacher | Manage many students      | Have examples compilable by a standard compiler on any system                                                           | 0  | 50  | 50  |

Basic defensive programming is directly expressible through any of the proposed contract facilities. Without a language feature, custom libraries must be used and that is not ideal.

Advanced features of contract lifecycle are more difficult to teach when not directly supported, and none of the proposals provide a complete solution for that behavior (see the broad discussion of what sorts of lifecycle considerations impact contract use), while only P1607 provides a way to do so at all at a non-global granularity.

|     |                                    |         |                      |                                                                            |   |    |    |
|-----|------------------------------------|---------|----------------------|----------------------------------------------------------------------------|---|----|----|
| 171 | <a href="#">teach.dumbstudents</a> | Teacher | Manage many students | Have examples that are easy to build without digression into build systems | 0 | 50 | 40 |
|-----|------------------------------------|---------|----------------------|----------------------------------------------------------------------------|---|----|----|

More advanced usages with P1429 or P1607 are clearly dependent on more complicated configuration. N4820 provides a simple set of flags (build modes) that would argue for being the easiest to teach and use.

Note that nothing in P1429 prevented the support for the same set of build modes in addition to the more specific semantic per level setting that it required.

|     |                                 |         |                               |                                                                                                                                                  |   |    |    |
|-----|---------------------------------|---------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|
| 172 | <a href="#">teach.teachable</a> | Teacher | Build layers of understanding | Have simple explanation of assertions and their use to support simple programming tasks, including debugging erroneous programs.                 | 0 | 50 | 50 |
| 173 | <a href="#">teach.layering</a>  | Teacher | Build layers of understanding | Support the ability for advanced uses of contracts to be distributed across many different courses in a C++-focused computer science curriculum. | 0 | 50 | 50 |

N4820 and P1429 provide the least advanced features, but all of the proposals allow for basic contract use to be done ( `default` level contract annotations) without any of the more advanced options that might be available.

|     |                                 |                    |                                         |                                                                                       |   |    |    |
|-----|---------------------------------|--------------------|-----------------------------------------|---------------------------------------------------------------------------------------|---|----|----|
| 174 | <a href="#">compiler.benice</a> | Compiler Developer | Deliver best experience to my customers | Maximize implementation freedom by limiting what is strictly required by the standard | 0 | 40 | 40 |
|-----|---------------------------------|--------------------|-----------------------------------------|---------------------------------------------------------------------------------------|---|----|----|

The explicit specification of build modes seems to be seen as highly restrictive by compiler vendors, and the less specific the standard is about that aspect of the facility the user base seems to be satisfied.

|     |                               |                    |                  |                                                               |   |    |    |
|-----|-------------------------------|--------------------|------------------|---------------------------------------------------------------|---|----|----|
| 175 | <a href="#">compiler.best</a> | Compiler Developer | Deliver the best | Have a clear and simple specification that meets a clear need | 0 | 15 | 75 |
|-----|-------------------------------|--------------------|------------------|---------------------------------------------------------------|---|----|----|



|                                                                                                                                                                                                                                                                                |                                           |                         |                                                                |                                                                                                      |    |    |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|-------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------|----|----|----|
| 186                                                                                                                                                                                                                                                                            | <a href="#">large.simulation.disable</a>  | Large Codebase Owner    | Allow simulation or post-mortem testing of known failure modes | Optionally disable checking on a subset of individual annotations                                    | 25 | 0  | 0  |
| 187                                                                                                                                                                                                                                                                            | <a href="#">large.simulation.enable</a>   | Large Codebase Owner    | Allow simulation or post-mortem testing of known failure modes | Optionally allow checking of a subset of individual annotations to fail and access its recovery path | 25 | 0  | 0  |
| 188                                                                                                                                                                                                                                                                            | <a href="#">large.simulation.ignore</a>   | Large Codebase Owner    | Allow simulation or post-mortem testing of known failure modes | Optionally allow checking of a subset of individual annotations to fail and continue failing         | 25 | 0  | 0  |
| Subsets of annotations can be called out with macro-based solutions, but not with the global controls of N4820 or P1429.                                                                                                                                                       |                                           |                         |                                                                |                                                                                                      |    |    |    |
| 189                                                                                                                                                                                                                                                                            | <a href="#">large.perfcontrol.build</a>   | Large Codebase Owner    | Manage performance cost                                        | Constrain the set of built time checks according to their performance overhead                       | 0  | 25 | 25 |
| 190                                                                                                                                                                                                                                                                            | <a href="#">large.perfcontrol.runtime</a> | Large Codebase Owner    | Manage performance cost                                        | Constrain the set of runtime checks according to their performance overhead                          | 0  | 50 | 50 |
| The language proposals had minimal control over build-time checking, but were focused on doing any constraining based on the cost of the check.                                                                                                                                |                                           |                         |                                                                |                                                                                                      |    |    |    |
| 191                                                                                                                                                                                                                                                                            | <a href="#">large.narrowing</a>           | Large Codebase Owner    | Tune contract width in complex system                          | Be able to narrow individual contract so it fails in testing not in production                       | 25 | 0  | 0  |
| A P1607 contract can have a macro to control individual semantics and go through a lifecycle where it is checked or enforced in testing but left ignored in production. Without control, this cannot be done with N4820 or P1429.                                              |                                           |                         |                                                                |                                                                                                      |    |    |    |
| 192                                                                                                                                                                                                                                                                            | <a href="#">embedded.nochecking</a>       | Small Machine Developer | Minimize executable footprint                                  | Remove all checking and diagnostic (eg. source location) overhead entirely from the final binary     | 25 | 50 | 50 |
| The language based proposals allow for the removal of checking without                                                                                                                                                                                                         |                                           |                         |                                                                |                                                                                                      |    |    |    |
| 193                                                                                                                                                                                                                                                                            | <a href="#">embedded.nologging</a>        | Small Machine Developer | Minimize executable footprint                                  | Remove all logging and diagnostic (but not checking) overhead from the final binary                  | 25 | 10 | 10 |
| 194                                                                                                                                                                                                                                                                            | <a href="#">embedded.minimize</a>         | Small Machine Developer | Minimize executable footprint                                  | Remove all but the most important diagnostic overhead from the final binary                          | 25 | 0  | 0  |
| None of the proposals allow for total removal of logging information from the generated code, while a macro based facility could support that option.                                                                                                                          |                                           |                         |                                                                |                                                                                                      |    |    |    |
| It would conceivable be possible for link time optimization to recognize that a violation handler made no use of source information/did no logging and then it would be able to source information as well, but this would have significant compile time overhead on a system. |                                           |                         |                                                                |                                                                                                      |    |    |    |
| 195                                                                                                                                                                                                                                                                            | <a href="#">wg21.everythingelse</a>       | Language Developer      | Interoperate with Contracts                                    | Have a clear way to understand how contracts will interact with the standard library                 | 0  | 0  | 0  |

|                                                                                                                                        |                                    |                    |                                                          |                                                                                                                                                                                                                                                                                           |   |   |   |
|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|--------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|
| 196                                                                                                                                    | <a href="#">wg21.otherfeatures</a> | Language Developer | Extend contracts beyond pre/post conditions on functions | Be able to use contract-like syntax on past or present runtime checkable language features such as switches, pattern matching, etc. or what might happen on signed integer overflow, etc. This might allow configuration of trapping, logging, or assuming in other areas of language UB. | 0 | 0 | 0 |
| None of the proposed solutions began integration with the rest of the standard, though they would have facilitated it in various ways. |                                    |                    |                                                          |                                                                                                                                                                                                                                                                                           |   |   |   |

## Conclusion

This is a lot of information to digest, so it might help to be able to get some overview numbers on how the different proposals relate to one another. There are endless ways to do this, and we make no attempt to be complete in this analysis, but we will present some approaches that might be useful.

- Simply adding up what percentage of the total value each proposal satisfies produces the following results:

| Proposal | Score   | Score W/Macros |
|----------|---------|----------------|
| N4842    | 0.000%  | 22.041%        |
| N4820    | 31.454% | 31.454%        |
| P1429    | 37.908% | 37.908%        |
| P1607    | 42.321% | 43.087%        |

- An alternative is to integrate the results of the initial polls that were reported in [P1995R0](#), using those to weight the scores provided above (i.e., the values in the "Score" column in that paper):

| Proposal | Score   | Score W/Macros |
|----------|---------|----------------|
| N4842    | 0.000%  | 24.683%        |
| N4820    | 38.306% | 38.306%        |
| P1429    | 44.703% | 44.703%        |
| P1607    | 48.847% | 49.527%        |

- Similarly, we could instead only consider the scores for use cases that had a score in P1995R0 above a certain threshold, such as everything with a score over 1.0:

| Proposal | Score   | Score W/Macros |
|----------|---------|----------------|
| N4842    | 0.000%  | 28.419%        |
| N4820    | 48.156% | 48.156%        |
| P1429    | 54.359% | 54.359%        |
| P1607    | 57.940% | 58.634%        |

Note that these are not intended to produce a complete measure of those proposals, but simply as an example of how they might be compared. These totals are not a good measure of which proposal is intrinsically better, but the changes to these totals from any individual proposal should be considered relevant when trying to understand how all users might benefit from any particular change.